1. Arsitektur & Hubungan Antar-Komponen Sistem

Bagaimana Web dan Android Saling Berkomunikasi?

Sistem ini tidak menggunakan server socket mandiri, melainkan bertumpu pada **Cloud Firestore** sebagai *event broker* realtime:

1. **Pelanggan Web (Laravel):** Mengirimkan data formulir pesanan melalui HTTP POST. Controller Laravel memformat dokumen NoSQL dan memanggil `FirestoreService->createOrder()` via Google Cloud REST API v1. Dokumen masuk ke koleksi `/orders` dengan field `status: "PENDING"` dan `source: "CUSTOMER_WEB"`.
2. **Android Gateway (MainActivity):** Memasang `addSnapshotListener` pada koleksi `/orders` dengan filter status PENDING. Saat dokumen web masuk, event `DocumentChange.Type.ADDED` terpicu secara instan (sub-second), membunyikan alarm bel lewat `SoundHelper` dan menampilkan dialog konfirmasi di layar kasir.
3. **Dapur KDS (AntrianFragment):** Kru dapur memproses pesanan dengan mengubah status ke `PREPARING`. Halaman web pelanggan yang sedang memantau secara berkala (polling API status) mendeteksi perubahan ini dan memperbarui indikator visual di browser pelanggan secara live.
4. **Serah Terima & Pengambilan (QRScanActivity):** Kasir memindai QR Code di layar pelanggan dengan kamera HP. Status pesanan diubah ke `COMPLETED` di Firestore, dan seluruh rincian transaksi otomatis disimpan permanen ke dalam tabel SQLite lokal `riwayat_transaksi` via `DBOpenHelper` sebagai arsip offline.

2. Bedah Kode Sisi Mobile Android — Gateway, POS & KDS

`MainActivity.kt` Core Gateway

app/src/main/java/com/pesenhub/jenggirat/MainActivity.kt • 604 baris

Untuk Apa? Menjadi kontainer utama dengan `BottomNavigationView` (Kasir, Antrean, Riwayat, Dashboard, Akun), memeriksa status verifikasi sesi email Firebase Auth, mendaftarkan Notification Channel sistem Android 13+, serta menjalankan background snapshot listener Firestore untuk mendeteksi pesanan web baru.
Kenapa Dibuat Begini? Jika listener Firestore dipasang di level Fragment, listener akan mati saat kasir berpindah tab. Meletakkan listener di `MainActivity` menjamin pesanan pelanggan web selalu tertangkap dan alarm bel selalu berbunyi di mana pun kasir sedang berada.

```
// Cuplikan Logika Kunci: Deteksi Realtime Web Order di Root webOrderListener = FirebaseFirestore.getInstance().collection("orders") .whereEqualTo("status", "PENDING") .addSnapshotListener { snapshots, error -> for (dc in snapshots.documentChanges) { if (dc.type == DocumentChange.Type.ADDED && isFromWeb) { SoundHelper.playBell() tampilkanDialogOrderBaru(dc.document) } } }
```

`KasirFragment.kt` POS Module

app/src/main/java/com/pesenhub/jenggirat/KasirFragment.kt • 698 baris

Untuk Apa? Modul POS kasir outlet yang menangani: (1) Pencarian nama martabak cepat via `AutoCompleteTextView`, (2) Filter kategori menu horizontal chips, (3) Modal kustomisasi topping/ekstra isian (keju mozzarella, sapi, ayam, dll), (4) Keranjang belanja melayang, (5) Pilihan metode pembayaran (Tunai/QRIS) via `RadioButton`, (6) Pemilihan estimasi jam siap ambil via `TimePickerDialog`, dan (7) Pengiriman order ke Firestore koleksi `/orders`.

Kenapa Dibuat Begini? Memenuhi checklist komponen interaktif UI PM & PML. Pemisahan input ke dalam 2 modal BottomSheet (Sheet Kustomisasi Menu & Sheet Checkout Bayar) mencegah tampilan layar menjadi sesak di layar smartphone vertikal.

```
// Cuplikan Logika Kunci: TimePickerDialog & Submit Order Kasir TimePickerDialog(requireContext(), { _, h, m -> estimasiJamAmbil = "Jam $h:$m" }, cal.get(Calendar.HOUR_OF_DAY), cal.get(Calendar.MINUTE), true).show()
dbFirestore.collection("orders").document(idPesanan).set(dataFirestore) .addOnSuccessListener { SoundHelper.playBell(); startActivity(intentDetail) }
```

AntrianFragment.kt & AntrianAdapter.kt KDS Dapur

app/src/main/java/com/pesenhub/jenggirat/AntrianFragment.kt • 370 baris

Untuk Apa? Layar KDS (Kitchen Display System) dapur outlet martabak. Memantau dokumen pesanan yang berstatus `PENDING`, `CONFIRMED`, dan `PREPARING` secara urut waktu (FIFO). Mengizinkan kru dapur mengubah status pesanan melalui: (1) `ContextMenu` saat item ditekan tahan (long-press), dan (2) `PopupMenu` saat tombol opsi ⋮ diklik.

Kenapa Dibuat Begini? Sesuai materi Bab V Modul PM (Menu & Dialog). Saat kru dapur menandai pesanan menjadi `COMPLETED`, fungsi `arsipKeSQLite()` otomatis dijalankan untuk menyalin data ke SQLite lokal sebelum pesanan hilang dari daftar antrean aktif.

```
// Cuplikan: ContextMenu & Otomatis Arsip ke SQLite saat Selesai override fun onContextItemSelected(item: MenuItem): Boolean { if (item.itemId == R.id.ctx_selesai) { updateStatusOrder(docId, order, "COMPLETED") // trigger arsip SQLite } }
```

DBOpenHelper.kt Local Database

app/src/main/java/com/pesenhub/jenggirat/DBOpenHelper.kt • 35 baris

Untuk Apa? Mewarisi `SQLiteOpenHelper` untuk mengelola database lokal smartphone kasir bernama `pesenhub_riwayat.db` dengan tabel `riwayat_transaksi` (id_pesanan, nama_pelanggan, hp, detail_item, metode_bayar, total_bayar, status_order, waktu_selesai).
Kenapa Dibuat Begini? Mengikuti konvensi Bab VIII Modul PM dosen (DML/DDL langsung via `execSQL` dan `rawQuery` tanpa Room library). Data pesanan yang sudah selesai bersifat final dan permanen, sehingga tidak perlu terus-menerus membebani kuota baca cloud Firestore.

```
// Cuplikan DDL Tabel SQLite val sqlRiwayat = "create table riwayat_transaksi(id_pesanan text primary key, nama_pelanggan text not null, hp_pelanggan text, detail_item text, metode_bayar text, total_bayar integer, status_order text, waktu_selesai text)" db?.execSQL(sqlRiwayat)
```

RiwayatFragment.kt & RiwayatAdapter.kt Laporan SQLite

app/src/main/java/com/pesenhub/jenggirat/RiwayatFragment.kt • 159 baris

Untuk Apa? Membaca arsip transaksi selesai dari SQLite lokal melalui `db.rawQuery("select * from riwayat_transaksi...", null)`, menghitung total omzet penjualan kotor outlet, dan menyediakan filter transaksi berdasarkan tanggal dengan `DatePickerDialog`.
Kenapa Dibuat Begini? Menjamin laporan penjualan outlet tetap dapat dibuka dan dihitung secara akurat meskipun outlet martabak sedang berada di lokasi tanpa koneksi internet sama sekali (*zero internet dependency*).

DashboardFragment.kt Analytics & Volley

app/src/main/java/com/pesenhub/jenggirat/DashboardFragment.kt • 166 baris

Untuk Apa? Menampilkan beranda ringkasan outlet: menghitung jumlah pesanan aktif (Firestore realtime), total transaksi selesai & omzet (SQLite lokal), info profil login, navigasi cepat antar menu, dan mengambil kutipan motivasi operasional melalui HTTP GET Request pustaka **Volley**.
Kenapa Dibuat Begini? Memenuhi checklist Volley HTTP Request (#19). Menggabungkan metrik cloud dan lokal dalam satu layar sehingga kasir memiliki visibilitas menyeluruh atas kondisi outlet.

```
// Cuplikan Volley HTTP Request val queue = Volley.newRequestQueue(requireContext()) val request = JsonObjectRequest(Request.Method.GET, "https://dummyjson.com/quotes/random", null, { res -> b.txQuoteText.text = res.optString("quote") }, { }) queue.add(request)
```

OrderDetailActivity.kt Struk & QR Generator

app/src/main/java/com/pesenhub/jenggirat/OrderDetailActivity.kt • 147 baris

Untuk Apa? Menampilkan rincian struk digital pesanan (nama pelanggan, daftar item, metode bayar, catatan) dan meng-*generate* gambar QR Code dari ID pesanan menggunakan pustaka ZXing `BarcodeEncoder().encodeBitmap(orderNumber, BarcodeFormat.QR_CODE, 400, 400)`.
Kenapa Dibuat Begini? Memenuhi checklist QR Code Generator (#25) sesuai Bab 10 Modul PM. QR code pada struk digital ini berfungsi sebagai token bukti fisik yang siap diverifikasi saat pelanggan datang ke kasir.

QRScanActivity.kt Hardware Kamera & QR Scan

app/src/main/java/com/pesenhub/jenggirat/QRScanActivity.kt • 151 baris

Untuk Apa? Membuka kamera gawai menggunakan ZXing `CompoundBarcodeView`, memindai QR Code di layar HP pelanggan, mencocokkan dokumen pesanan di Firestore, dan menyediakan tombol verifikasi untuk mengubah status menjadi `COMPLETED` serta mengarsipkannya ke SQLite.

Kenapa Dibuat Begini? Memenuhi checklist Hardware Kamera & Scanner QR (#20 & #25). Menghilangkan potensi human error kasir saat mencari pesanan di tumpukan antrean manual.

```
// Cuplikan Verifikasi Scan Kamera b.barcodeScannerView.decodeSingle { result ->
dbFirestore.collection("orders").document(result.text).get() .addOnSuccessListener { doc -> tampilkanHasilVerifikasi(doc) } }
```

FormMenuActivity.kt & KelolaMenuActivity.kt CRUD Menu Cloud

app/src/main/java/com/pesenhub/jenggirat/FormMenuActivity.kt • 212 baris

Untuk Apa? Menangani operasi CRUD katalog martabak langsung ke Cloud Firestore koleksi `/menus`: menambah item, mengubah harga per channel (kasir vs web pelanggan), mengubah status ketersediaan stok menu (available true/false), dan memilih gambar menu.

Kenapa Dibuat Begini? Memastikan katalog produk yang tampil di Website Pelanggan selalu sinkron secara otomatis saat admin outlet mengubah harga atau mematikan stok yang sedang habis di gerai.

UploadMenuActivity.kt & SupabaseStorageHelper.kt Kamera/Galeri + CDN

app/src/main/java/com/pesenhub/jenggirat/SupabaseStorageHelper.kt • 130 baris

Untuk Apa? Mengambil foto martabak baru langsung dari Kamera gawai atau Galeri, mengompresnya menjadi byte array, lalu mengunggahnya secara asinkron ke bucket Supabase Storage via REST HTTP API.
Kenapa Dibuat Begini? Memenuhi checklist Kamera (#20), Galeri (#26), dan Supabase (#33). Firestore tidak efisien dan boros kuota jika dipaksa menyimpan binary base64 gambar. CDN Supabase menghasilkan link URL publik yang ringan dibuka oleh browser web maupun aplikasi mobile.

```
// Cuplikan REST Upload Supabase val conn =
URL("$SUPABASE_URL/storage/v1/object/$BUCKET_NAME/menu-images/$filename").openConnection() as HttpURLConnection
conn.requestMethod = "POST"; conn.setRequestProperty("apikey", SUPABASE_KEY) conn.outputStream.write(imageBytes) // Menghasilkan URL publik CDN
```

MapsActivity.kt GPS & Peta OSMdroid

app/src/main/java/com/pesenhub/jenggirat/MapsActivity.kt • 141 baris

Untuk Apa? Menampilkan peta interaktif OpenStreetMap menggunakan pustaka **OSMdroid** (`MapView`, `Marker`), menandai koordinat titik outlet Martabak Jenggirat Kediri (-7.8166, 112.0116), dan melacak lokasi real kasir via `FusedLocationProviderClient` (GPS).
Kenapa Dibuat Begini? Memenuhi materi Bab 13 Modul PM dosen (GPS & Peta). Menggunakan OSMdroid yang bersifat *open-source* dan bebas biaya token API Google Maps.

LoginActivity.kt & RegisterActivity.kt Firebase Auth & Session

app/src/main/java/com/pesenhub/jenggirat/LoginActivity.kt • 210 baris

Untuk Apa? Otentikasi staf gerai menggunakan **Firebase Authentication** (`signInWithEmailAndPassword`, `createUserWithEmailAndPassword`), memeriksa verifikasi email pengguna, dan menyimpan username di `SharedPreferences` saat opsi "Ingat Username" dicentang.

Kenapa Dibuat Begini? Memenuhi Bab 04 PML (Firebase Auth) dan Bab 07 PM (SharedPreferences). Mengamankan menu kelola kasir dan memudahkan proses login harian staf gerai.

SoundHelper.kt & PesenHubFCMService.kt Audio Alert & Push FCM

app/src/main/java/com/pesenhub/jenggirat/SoundHelper.kt • 40 baris

Untuk Apa? Memutar nada dering bel pesanan menggunakan `MediaPlayer` (Checklist #24) dan menangani penerimaan pesan push notification dari server Firebase Cloud Messaging (Checklist #31).

Kenapa Dibuat Begini? Di dapur yang bising saat memasak martabak, staf tidak dapat selalu menatap layar smartphone. Audio alert menjamin pesanan web yang baru masuk langsung ditangani kru dapur tanpa tertunda.

3. Bedah Kode Sisi Web Pelanggan (Laravel PHP — pesenhub_web/)

routes/web.php & routes/api.php Routing Layer

pesenhub_web/routes/web.php • 30 baris

Untuk Apa? Mendefinisikan titik masuk URL pelanggan: `GET /` (katalog menu), `POST /pesan` (simpan order), `GET /pesanan/{id}` (halaman status), `GET /lokasi` (info outlet), serta endpoint API `GET /api/pesanan/{id}/status` untuk polling realtime.

Kenapa Dibuat Begini? Memisahkan jalur tampilan HTML Blade dari endpoint API JSON yang dikonsumsi secara asinkron oleh skrip polling JavaScript frontend pelanggan.

CustomerAuthController.php Guest Session

pesenhub_web/app/Http/Controllers/CustomerAuthController.php • 71 baris

Untuk Apa? Menyimpan sesi identitas tamu pelanggan (Nama & Nomor WhatsApp) ke dalam session server Laravel dengan masa kedaluwarsa 5 jam.

Kenapa Dibuat Begini? Menghilangkan friksi pendaftaran akun. Pelanggan restoran tidak suka mendaftar email/password hanya untuk memesan martabak. Sesi 5 jam cukup untuk memesan dan melacak status antrean hingga selesai makan.

CustomerOrderController.php Core Controller

pesenhub_web/app/Http/Controllers/CustomerOrderController.php • 250 baris

Untuk Apa? (1) Mengambil menu aktif dari Firestore lewat `getMenus()`, (2) Memvalidasi keranjang belanja JSON dan opsi topping, (3) Menghitung total harga & subtotal secara aman di sisi server, (4) Mengirimkan order ke Firestore via `createOrder()`, dan (5) Melayani API status pesanan untuk polling browser.
Kenapa Dibuat Begini? Semua kalkulasi harga dikerjakan ulang di server untuk mencegah manipulasi nominal harga belanja oleh pengguna melalui inspect element browser.

```
// Cuplikan Simpan Dokumen Pesanan ke Firestore $orderData = [ 'orderNumber' => 'ORD-WEB-' . date('Ymd-His') . '-' . rand(100, 999), 'status' => 'PENDING', 'source' => 'CUSTOMER_WEB', 'total' => $totalPrice, 'items' => $items, ]; $this->firestore->createOrder($orderData);
```

FirestoreService.php Firestore REST Client

pesenhub_web/app/Services/FirestoreService.php • 454 baris

Untuk Apa? Membangun komunikasi langsung ke Google Cloud Firestore REST API v1 menggunakan Laravel `Http::get`, `Http::post`, dan `Http::patch`. Bertanggung jawab mem-parsing struktur format dokumen Firestore (seperti `stringValue`, `integerValue`, `mapValue`, `arrayValue`) menjadi array PHP murni, dan sebaliknya saat mengirim data.
Kenapa Dibuat Begini? Keputusan Kunci: Pustaka resmi Google Cloud PHP SDK memerlukan ekstensi binary C (gRPC) yang sering kali tidak didukung atau sulit diinstal pada lingkungan web hosting standar. Menggunakan REST API murni menjamin web Laravel dapat berjalan di hosting apa pun tanpa dependensi C tambahan.

```
// Cuplikan Parser Firestore REST public function parseValue(array $val) { if (isset($val['stringValue'])) return $val['stringValue']; if (isset($val['integerValue'])) return (int) $val['integerValue']; if (isset($val['mapValue'])) { ... } }
```

BarcodeGeneratorService.php QR Code Generator

pesenhub_web/app/Services/BarcodeGeneratorService.php • 65 baris

Untuk Apa? Menggunakan pustaka `chillerlan/php-qrcode` untuk merender ID pesanan web menjadi gambar QR Code berbasis SVG atau Base64 Image string.
Kenapa Dibuat Begini? QR code yang dihasilkan ditampilkan di halaman bukti pesanan pelanggan (`status.blade.php`) agar siap dipindai oleh kasir Android saat pengambilan martabak.

> **`views/customer/menu.blade.php & status.blade.php Frontend UI`**

> pesenhub_web/resources/views/customer/ • Mobile-First Blade Views

> **Untuk Apa?** `menu.blade.php` menyajikan katalog martabak, modal pemilihan ekstra topping, dan keranjang belanja melayang

> (floating cart). `status.blade.php` menyajikan timeline visual status pesanan dengan script JavaScript polling berkala setiap 5 detik

> ke endpoint API status.

> **Kenapa Dibuat Begini?** Memungkinkan halaman pelacakan pesanan pelanggan di browser berubah secara live (misal: dari

> *Menunggu* menjadi *Sedang Dimasak*) saat kru dapur menekan tombol di HP Android tanpa perlu reload halaman manual.